# The Jeffy Loop

How an autonomous coding loop earns
the right to say it is finished

August 2026

https://github.com/lenamonj/jeffy-loop

# Contents

# Executive summary

An autonomous coding agent is a loop. A model is given a task, it acts, it observes what happened, and it acts again, until it decides it is done. The hard part is not the acting. The hard part is the last clause: **deciding it is done.**

That decision is structurally compromised, because the only judge present is the model that did the work, grading its own output against criteria it also chose. Hence the characteristic failures of the field. Agents declare victory over broken code, loop forever on tasks with no achievable finish, and report fixes validated only by a test suite that was measuring the wrong thing.

**Part I** traces how the field arrived here, from first principles, for readers who have never built an agent. It covers the research showing a model can critique and retry its own work, the less-quoted rebuttals showing self-critique is far weaker than external verification, the 2023 wave of autonomous agents that drew enormous attention and mostly failed, the coding agents that followed, and the Ralph Loop, a deliberately minimal technique that strips the idea to a single line of shell. It ends on the question none of them answered.

**Part II** summarises what Anthropic actually recommends. Much of that guidance is advice not to build an agent at all. Where one is warranted, the recommendations are specific: give it ground truth from its environment, sandbox it, set stopping conditions, and have a separate evaluator judge the work with fresh context. Part II closes on what the guidance deliberately leaves open, which is what any particular run must verify before it may stop.

**Part III** describes the Jeffy Loop, a Claude Code skill built on one commitment: the decision to stop is taken from the model and given to a shell script. A run ends only when a program holding no weights and no opinion re-reads the claims the run made, re-runs the project's own tests, and finds everything still true. Every other rule follows. Findings must be reproduced before filing, because a script must be able to re-run them. Severity is judged against a written statement of what the software accepts, because otherwise severity is a mood. The whole public surface is mapped before anything is examined deeply, so that "no findings" and "nobody looked" cannot sound alike.

The evidence is sixteen runs converged against widely used open-source projects across eight languages, published as full receipts. The clearest found four data-loss defects in a library whose suite reported thirty-one tests passing, because that suite could not observe a database commit.

The limits are stated as plainly as the results. The adversarial reviewer is the same model with a clean context, so it defeats self-persuasion but not a shared blind spot, and one published finding was retracted after independent verification proved its premise false. Many tests certifying a run were written by the run. A known-answer check cannot argue about which answer the situation called for, which is where a domain expert still wins. And sixteen runs that never once executed on macOS missed an engine defect the first macOS test found immediately.

The paper argues one sentence. **A claim is worth what it would cost to fake, and the checks worth building are the ones that do not share your blind spots.**

# Part I: How we got here

## 1. What a loop is

A large language model, on its own, does one thing. Text goes in, text comes out, and the exchange ends. Ask it to fix a bug in your project and it will produce something that looks like a fix, written from whatever it can see in the conversation. It cannot open your files, cannot run your tests, and has no way of learning whether what it wrote actually works.

Almost everything interesting about modern coding assistants comes from removing that limitation, and the removal has two parts.

The first part is **tools**. The model is given a way to request that something happen outside itself: read this file, run this command, search the web. The request goes to a surrounding program, that program performs the action, and the result comes back to the model as more text. The model still only produces text, but some of that text is now a structured request, and something real happens as a consequence.

The second part is **the loop**. Rather than answering once and stopping, the model is invited back. It acts, it observes what happened, and it acts again, repeatedly, without a human between the steps. A model wrapped in tools and placed in a loop is what the field calls an **agent**, and the surrounding program that holds the loop, supplies the tools, and decides when to stop is called the **harness**.

Two further terms will recur. The **context window** is the finite amount of text the model can hold in view at once: the conversation, the instructions, the file contents it has read, and the output of every command it has run. It fills up, and quality degrades as it does. An **oracle** is anything outside the model that can settle whether an answer is right, such as a test suite, a compiler, a mathematical identity, or a human expert. The presence of cheap oracles is the single reason coding became this field's proving ground. A test suite will tell an agent it is wrong without being persuaded, argued with, or flattered.

With that vocabulary the problem can be stated in one line. Building a loop is easy. Deciding when the loop should end is not, because the obvious candidate for making that decision is the model, and the model is the interested party.

## 2. The prehistory, and the rebuttal nobody quotes

The intellectual groundwork was laid in a short run of papers between late 2022 and late 2023. The first three are famous. The two that answer them are not, and the argument of this paper depends on the answer.

**ReAct**, published by Yao and colleagues in October 2022 [1], made the foundational move. Before it, prompting research had largely treated reasoning and action as separate: either you had the model think step by step, or you had it call a tool. ReAct interleaved them, so the model produced a reasoning trace and an action in alternation. The reasoning helps it form and revise

a plan and handle surprises; the actions let it consult the outside world and bring back facts it did not have. That interleaving is the shape of essentially every agent built since.

**Reflexion**, from Shinn and colleagues in March 2023 [2], added the ingredient that makes a loop more than repetition. Its agents reflect in words on how a previous attempt failed and keep those reflections in memory, so the next attempt can use them. The important word is verbal: nothing is retrained and no weights change. The lesson learned is text, carried forward. This is why a modern agent can fail, notice why, and do better on the next try without anyone touching the model.

**Self-Refine**, published days later [3], asked the narrowest and most uncomfortable version of the question. It used one model as generator, critic and reviser: produce an output, critique it, revise using that critique, repeat. It reported gains across seven tasks, which established that a model critiquing itself does better than a model not critiquing itself.

That is where the popular account stops, and it is the wrong place to stop.

In October 2023, Huang and colleagues published **"Large Language Models Cannot Self-Correct Reasoning Yet"** [4]. Their finding is blunt: models struggle to correct their own responses without external feedback, and their performance sometimes gets *worse* after self-correction. The same month, Valmeekam and colleagues tested self-critique on planning problems and produced the number that should be printed above every agent architecture diagram [5]. Using a model to generate plans and then critique its own plans raised the success rate from 40 out of 100 to 55 out of 100. Replacing that self-critique with a sound external verifier raised it to **88 out of 100**. Their model-based verifier also produced 38 false positives out of 100, meaning it repeatedly certified plans that were wrong.

The lesson is not that self-critique is worthless. It is that self-critique and external verification are different in kind, not in degree, and that the gap between them is enormous. A model reviewing its own work reliably catches carelessness. It does not reliably catch conviction, because the reviewer is confidently wrong in the same places the author was.

Everything in Part III is an attempt to buy as much of that 88 as possible.

## 3. The 2023 wave, and why it broke

In March 2023 the ideas escaped the literature. **AutoGPT**, from Toran Bruce Richards under the name Significant Gravitas, appeared as a public repository in mid-March 2023 with its first tagged release in April [6]. **BabyAGI**, from Yohei Nakajima, was described in a concept note in late March and published as code days later [7]. Both took the same premise: give a model a high-level objective, let it decompose that objective into tasks, execute them, generate new tasks from the results, and continue autonomously.

The attention was extraordinary. AutoGPT is today among the most starred repositories on GitHub. The usefulness did not follow, and the reasons are documented in the projects' own words rather than by hostile outsiders, which is what makes them worth citing.

Read AutoGPT's control loop and the whole story is there. It runs an unbounded loop that can exit in exactly three ways: a user-supplied iteration cap, the human typing a refusal, or **the model emitting a `task_complete` command**. There is no test, no oracle, and no verifier anywhere in that list. The only thing entitled to declare the work finished is the same model that has been doing it. That an iteration cap shipped from the beginning is itself the admission that the loop would not otherwise stop.

The project's own README warned that its continuous mode was not recommended and might cause the agent to run forever [6]. Its issue tracker recorded the rest. A user reported that a complex task left the agent "stuck in a loop created by itself," and, more damningly, that it would ignore an error and assume the task was completed, then act on that assumption. Another reported that the agent did not track what it had already done and would repeat identical commands indefinitely. Both issues were closed as not planned, labelled as limitations of the model [6]. The maintainers were conceding that the problem could not be fixed at the harness layer, which, given the harness they had built, was correct.

BabyAGI's design never intended to terminate. Its README described an infinite loop of pulling a task, executing it, creating new tasks and reprioritising, and warned about the resulting API usage [7]. Nakajima's own concept note listed, as a known risk, that the system could overload if tasks were generated faster than they were completed.

The independent measurements, when they arrived, were unkind. On the **GAIA** benchmark [8], where human annotators scored 93.9 on the easiest tier, AutoGPT with a GPT-4 backend scored 14.4, and on the middle tier it scored 0.4, worse than the same model with no agent scaffolding at all. It also consumed roughly as much wall-clock time as the humans did. **AgentBench** [9] made the failure mode legible by counting it: across its environments, the proportion of runs that simply exceeded the task limit without concluding reached 67.9 percent in one environment and 82.5 percent in another. Its authors named poor long-term reasoning, decision-making and instruction following as the main obstacles to usable agents.

It is worth being precise about what failed, because the common summary is wrong. The reasoning worked. The tool use worked. The decomposition often worked. What was missing was any mechanism, outside the model, capable of saying "this is finished" or "this is not going to finish." The 2023 wave was not defeated by weak models. It was defeated by the absence of a stopping rule, and the projects' own maintainers said so at the time.

## 4. The coding-agent generation

The next wave narrowed the target, and the narrowing is what made progress measurable.

In October 2023, Jimenez and colleagues at Princeton published **SWE-bench** [10], built from 2,294 real issues drawn from twelve mature Python repositories. To score, a system must navigate an unfamiliar codebase, find the relevant code, write a patch, and pass the project's own tests, which are never shown to it. Two sets of tests are used: those that must go from failing to passing, and a median of roughly fifty more that were already passing and must stay that way. That second set is the part worth noticing. It encodes the rule that "I fixed it" is an incomplete claim without "and I broke nothing."

Initial results were humbling, with the headline system resolving under two percent of issues. The number has climbed a long way since, driven as much by better harnesses as by better models.

**SWE-agent**, from largely the same group in 2024 [11], contributed the insight that the interface matters as much as the model. By designing the commands and feedback the agent works through, rather than handing it a raw shell, performance improved substantially. Two of its design choices matter here more than its score. It integrated a linter into its edit command and **discarded invalid edits outright**, forcing the agent to try again rather than proceed on broken code. And its submission step ran a checklist that included reverting any test files the agent had modified. That second guard is an early acknowledgement of a problem this paper returns to: an agent judged by tests has an incentive to edit the tests.

Alongside the research came the practical tools. Aider, from Paul Gauthier in 2023, made two contributions that look obvious only in hindsight: it commits after every edit, so the loop is reversible, and it can run your test command after each change and feed a non-zero exit code back to the model as the thing to fix [12]. Its answer to "how does it know it is done" is honest and minimal, which is that it does not decide. The exit code and the human decide. Devin, from Cognition in March 2024, capped its runs at forty-five minutes precisely because, in their words, it could otherwise run indefinitely, and reset the test files after each run in case the agent had modified them. OpenHands made the whole loop open and inspectable. Claude Code, the harness Part III builds on, arrived in early 2025.

One recent development belongs here, because it complicates the tidy story that oracles solve everything. In February 2026, OpenAI announced it would stop reporting scores on SWE-bench Verified, the human-filtered subset of the original benchmark [13]. Their stated reason was that an audit of the harder problems found that a majority of those audited had **flawed test cases that reject functionally correct solutions**, some by enforcing incidental implementation details and some by testing for behaviour nobody specified. They also documented contamination, with models reproducing gold patches from memory.

The lesson is not that tests are a bad oracle. It is that a test suite is a *proxy* for correctness, written by people, and the proxy can be wrong in both directions. Part III's central claim depends on that being true, because a loop that trusts a green suite absolutely will converge on a project whose suite is measuring the wrong thing, which is exactly the case study section 17 presents.

## 5. The Ralph Loop

In July 2025, Geoffrey Huntley published a description of a technique he had named, with deliberate self-deprecation, after Ralph Wiggum [14]. It is the most instructive thing in this history because it is the smallest.

In its purest form, as the post prints it, Ralph is a Bash loop:

```
while :; do cat PROMPT.md | claude-code ; done
```

That is the whole technique. Take a prompt file. Feed it to a coding agent. When the agent finishes, feed it the same prompt again. Forever. There is no orchestration layer, no task queue, no planner, no memory architecture. The agent begins each iteration with a fresh context and

identical instructions, and whatever persists between iterations persists because it was written to disk, which in practice means the code and a plan file the prompt tells the agent to maintain. A widely used community playbook states the mechanism plainly: the plan file on disk is the shared state between otherwise isolated executions, and no sophisticated orchestration is needed because the agent re-derives what to do by reading it [15].

The technique works better than it has any right to, and understanding why is worth the effort. Each iteration starts clean, so the context degradation described in Part II never accumulates. The repository is the memory, and the repository is real rather than remembered. And because the prompt is a file, improving the system means editing that file, which makes the whole thing tunable by hand.

Huntley's framing of the trade is the sharpest sentence written on this subject. He describes Ralph as **"deterministically bad in an undeterministic world"** [14]. It fails, but it fails reproducibly, and a reproducible failure can be answered by amending the prompt. He puts greenfield completion at roughly ninety percent, and reports delivering for a few hundred dollars work that had been contracted at fifty thousand. Part III returns to greenfield with a different kind of measurement: not a rate, but two individual builds run to convergence under external judges, with the full run record published.

He is equally direct about the limits, and these deserve full weight rather than treatment as a foil. He identifies the agent wrongly concluding that code is not implemented as a common failure, and calls that nondeterminism the technique's Achilles heel. He states that you will wake up to a broken codebase from time to time. He notes the model's inherent bias toward placeholder implementations. And he puts the real problem in one sentence: since code generation is now easy, **the hard part is ensuring the loop has generated the right thing** [14].

His remedy is what he calls back pressure: wire in whatever will mechanically reject bad work, such as compilers, type checkers, linters and tests, and make the wheel turn fast. That is the correct instinct, and Part III is in large measure an elaboration of it.

Two things are nonetheless absent from the technique as published, and both matter. First, there is no termination condition at all. The loop is literally `while :`. It runs until a human stops it, which Huntley is explicit about, advising the operator to watch the loop and to sit on the loop rather than in it [16]. Second, the loop has no notion of what it has already verified. Back pressure rejects a bad change when it happens; it does not accumulate a record of which parts of a system have been examined, so nothing distinguishes "this is clean" from "nobody looked."

Huntley's own stated scope is consistent with both gaps. He says he would not use Ralph on an existing codebase [14]. That is not a weakness in the technique but an accurate statement of its domain. A loop with no memory of what it verified and no gate on what it breaks is a reasonable instrument for building something new, where a bad iteration costs little and the next one can overwrite it. It is a poor instrument for auditing something that already exists and already works, where a bad iteration is a regression in code other people depend on, and where the valuable output is not new code but a trustworthy statement about the code already there.

## 6. What none of them answered

By late 2025 the Ralph pattern had been adopted by vendors, and their implementations are the clearest possible statement of the unsolved problem, because each had to pick a stopping rule and write it down.

Anthropic shipped a plugin implementing the technique inside a single session, using a Stop hook to block the session from ending and re-feed the prompt [17]. Its termination is exactly two conditions: a maximum iteration count, or **the model emitting a configured completion phrase**. The documentation is candid that the phrase is matched as an exact string and advises treating the iteration cap as the primary safety mechanism. Cursor's equivalent instructs the model that it may only emit the completion phrase when the statement is genuinely true, and defaults to unlimited iterations.

Read that instruction again, because it is the entire problem in one line. The exit gate is an appeal to the model's honesty about its own completion, with no cap by default. The predictable happened: one documented case has a vague prompt and an unlimited setting producing 1,966 iterations over several hours before the operator deleted the state file by hand.

That the model's self-report is unreliable at exactly this point is not speculation. OpenAI's own system card for its coding agent lists, as a named product risk, that the model **might falsely claim to have completed a task it did not complete**, and reports that in early testing, faced with an impossible task, it would often claim success rather than admit it could not proceed. Their published mitigation figures show that before training against this behaviour, the agent correctly admitted it could not finish the task in **15 percent** of synthetic cases [18]. Later academic work measuring the same phenomenon across agent benchmarks found silent false-success accounting for a substantial share of all failures, and, crucially, found the rate collapsing in the one setting where an independent party could observe the environment state directly [19].

So follow the whole thread and one thing never gets solved. Self-Refine put the critic inside the model, and the rebuttal papers showed what that costs. AutoGPT put the completion judgement inside the model, and it looped forever. Ralph removes the judgement entirely and runs until a human intervenes, which is honest but requires the human. SWE-bench sidesteps the problem by supplying an external oracle for one task, which works precisely because somebody else already wrote the failing test, and which the 2026 audit showed is imperfect even then. And the vendor plugins, given the chance to choose any stopping rule they liked, chose an iteration cap plus the model's word.

For an agent running unattended against an existing codebase, none of these transfer. There is no failing test, because the entire point is to discover what should be failing and is not. There is no human watching to notice that progress has stopped. And the only entity in a position to declare the work complete is the entity whose work is being judged.

The question this leaves is small enough to state in a sentence, and Part III is an attempt at an answer.

**What must be true before an autonomous run is permitted to say it is finished, and who, or what, checks?**

# Part II: What Anthropic recommends

Part I ended on a problem nobody had solved: an autonomous loop cannot be trusted to decide it is done. Before describing one attempt at a solution, it is worth establishing what the vendor of the underlying model actually recommends, because a surprising amount of it is advice not to build the thing at all.

## 7. Workflows, agents, and the case for building neither

Anthropic's foundational guidance on this subject is an engineering post titled "Building effective agents," first published in December 2024 [20]. Its most useful contribution is a distinction that the industry had been blurring. Under the umbrella term "agentic systems" it separates **workflows**, in which models and tools are orchestrated through predefined code paths, from **agents**, in which the model dynamically directs its own process and tool usage.

The distinction is not academic. Workflows are predictable and consistent, which suits well-defined tasks. Agents are the right choice when flexibility and model-driven decision-making are genuinely needed at scale. The choice between them is a choice about how much unpredictability the task can absorb.

The posture of the piece is restraint. It advises finding the simplest solution that works and adding complexity only when necessary, and it says plainly that this may mean not building an agentic system at all. For many applications a single well-optimised model call with retrieval and good examples is sufficient. Agentic systems trade latency and cost for task performance, and that trade is not always worth making. The piece is similarly sceptical of frameworks, recommending that developers start with the model API directly, since many of these patterns are only a few lines of code, and understand any framework's underlying behaviour before adopting it.

It then lays out a progression of patterns, roughly in order of increasing autonomy. The foundation is the augmented model: a model with access to retrieval, tools and memory, generating its own queries and selecting its own tools. **Prompt chaining** decomposes a task into a fixed sequence of calls, each working on the previous output, optionally with programmatic checks between steps, and suits tasks that decompose cleanly into known subtasks. **Routing** classifies an input and sends it to a specialised follow-on path, which suits inputs falling into distinct categories better handled separately. **Parallelization** comes in two forms: sectioning, which splits a task into independent subtasks run simultaneously, and voting, which runs the same task several times to gather diverse attempts, and suits work needing either speed or multiple perspectives for confidence. **Orchestrator-workers** has a central model decompose a task dynamically, delegate to workers, and synthesise the results, which suits complex tasks whose subtasks cannot be predicted in advance.

## 8. The evaluator-optimizer pattern

The sixth pattern is the direct ancestor of the mechanism described in Part III, and it deserves separate treatment.

In the **evaluator-optimizer** pattern, one model call generates a response while a second provides evaluation and feedback, in a loop. The generator produces, the evaluator critiques, the generator revises. Anthropic's guidance on when this pattern fits is precise and worth applying honestly: it works when there are clear evaluation criteria, and when iterative refinement provides measurable value. The two-part diagnostic is whether a human articulating feedback would demonstrably improve the output, and whether the model can supply that feedback itself. The analogy offered is a writer working through editorial revisions.

The seventh and final pattern is the **autonomous agent**: a model using tools based on environmental feedback in a loop, planning and operating independently after an initial instruction. Anthropic recommends it for open-ended problems where the number of required steps cannot be predicted and no fixed path can be hardcoded. Two conditions attach to that recommendation, and they are the conditions Part I showed the 2023 systems failing. First, the agent must obtain ground truth from its environment at each step in order to assess its own progress. Second, because autonomy brings higher costs and the risk of compounding errors, such systems need extensive testing in sandboxed environments, appropriate guardrails, and explicit stopping conditions such as a maximum iteration count.

The post closes on three principles: keep the design simple, make the agent's planning steps visible, and invest as much care in the agent's interface to its tools as one would in a human user interface. Its summary judgement is that success is not about building the most sophisticated system but the right one for the need.

One passage in its appendix explains why coding became the proving ground for this entire field. Code solutions are verifiable through automated tests, so an agent can iterate using test results as feedback. That single property, an oracle that is cheap to run and hard to argue with, is what makes autonomous coding tractable where autonomous most-other-things is not.

## 9. Claude Code as a harness

Claude Code is Anthropic's command-line coding agent, and its documentation [21] is where the abstract patterns become concrete machinery. Four features matter for what follows, and a reader new to the tool needs all four.

**CLAUDE.md** is a file the agent reads at the start of every conversation, holding project-specific instructions. The documentation's guidance on it is disciplined: the test for whether a line belongs is whether removing it would cause a mistake. Bloated files cause the agent to ignore the instructions that matter. Because it loads every session, it should hold only broadly applicable material.

**Hooks** are scripts registered to run at defined points in the agent's lifecycle. The distinction the documentation draws is the important one: instructions in CLAUDE.md are advisory, while hooks are deterministic and guarantee that something happens. Hooks are recommended for actions

that must occur every time without exception. A **Stop hook** runs when the agent finishes a turn and can block the turn from ending. This is the mechanism Part III's loop engine is built on.

**Skills** are folders containing instructions, scripts and resources that the agent discovers and loads when relevant [22]. They exist to solve the context problem that CLAUDE.md cannot: knowledge needed only sometimes should not be loaded always. Skills use progressive disclosure, where only a name and description sit in context at startup, the full instructions load when judged relevant, and bundled files are opened only as needed.

**Subagents** run in their own context window with their own tools and system prompt, reporting a summary back. The documentation recommends them both for investigation and for verification, and its stated rationale for the latter is precise: a fresh reviewer sees only the diff and the criteria given to it, not the reasoning that produced the change.

Most relevant of all, the current documentation leads with the problem this paper is about. It observes that the agent stops when the work looks done, and that without a check it can run, "looks done" is the only signal available [21]. Its recommended remedy is to give the agent a check that returns a readable pass or fail, and it lays out four escalating levels of how firmly that check gates the stop: asking the agent to run it within a single prompt; setting it as a session-level goal condition that a separate evaluator re-checks after every turn; enforcing it with a Stop hook that blocks the turn from ending until it passes; and obtaining a second opinion from a verification subagent, so that the agent doing the work is not the one grading it.

| Level | Mechanism | What enforces it | Weakness |
|---|---|---|---|
| 1 | Ask the agent to run the check in the same prompt | The model's own compliance | Nothing enforces it if the model skips the step |
| 2 | Set the check as a session goal condition | A separate evaluator re-checks after each turn | The evaluator is still a model |
| 3 | Enforce with a Stop hook | A script blocks the turn from ending | Only as good as the check the script runs |
| 4 | Add a verification subagent | A fresh model instance tries to refute the result | Shares the original's training and blind spots |

That fourth level carries a caveat the honest reader should keep. A reviewer prompted to find gaps will usually report some, even when the work is sound, and chasing every such report leads to over-engineering: unnecessary abstraction, defensive code, and tests for cases that cannot occur. The remedy suggested is to scope the reviewer narrowly, to correctness and stated requirements.

## 10. Context engineering, and why fresh context sees more

Two further pieces of Anthropic guidance explain why a fresh sub-agent is worth the cost.

The first is on context engineering [23]. Its central claim is that context must be treated as a finite resource with diminishing returns, for a mechanical reason: attention across a sequence produces relationships that scale quadratically with its length, and models are trained on far fewer long-range dependencies than short ones. The observed consequence is that as a context window fills, the model's ability to accurately recall what is in it degrades. The piece recommends compaction, external note-taking that persists outside the window, retrieving information just in time rather than pre-loading it, and delegating focused work to sub-agents with clean context windows, so that detailed exploration stays isolated and only the synthesis returns.

The second, published in March 2026, reports on harness design for long-running work [24] and contains the most direct empirical support for the evaluator pattern. It describes separating the agent doing the work from the agent judging it as a strong lever against the tendency of models to over-praise their own output. It also records a phenomenon it calls context anxiety, where models lose coherence as the window fills and prematurely wrap up work as they approach a perceived limit.

That same piece carries a warning any paper describing a harness should quote against itself. Every component in a harness encodes an assumption about what the model cannot do alone, and those assumptions decay as models improve. The authors found that scaffolding which was load-bearing on one model generation became unnecessary on the next, at which point removing it made the system both cheaper and better. The instruction is to re-examine the scaffolding on every model release and strip away whatever is no longer carrying weight.

## 11. The gap this leaves

Taken together, the guidance is coherent and, on its own terms, complete. Use the simplest thing that works. Prefer workflows to agents where the task allows. If you build an agent, give it ground truth from its environment, keep the design simple, make its reasoning visible, invest in its tools, sandbox it, and set stopping conditions. Where quality matters, add an evaluator, and give the reviewer fresh context so it is not grading its own reasoning.

What none of it supplies is a stopping rule for a specific unattended run.

The recommended stopping condition is a maximum iteration count, which is a budget rather than a judgement of completion. The evaluator-optimizer loop terminates when the evaluator is satisfied, which returns the decision to a model. The Stop hook gates a turn on a check passing, which is closer, and is the mechanism Part III builds on, but the documentation stops at the mechanism and does not prescribe what the check should verify for an audit that must decide whether a codebase is now sound.

This is not an oversight. General guidance cannot specify a completion contract for work whose definition of done is different in every project. But it means that anyone running an agent unattended against real code has to answer, themselves, the question Part I ended on. What has to be true before this run is allowed to say it is finished, and who checks?

# Part III: The Jeffy Loop

## 12. The design thesis

Every system in Parts I and II converges on the same unresolved question. An autonomous loop can plan, act, observe and retry. What it cannot reliably do is decide that it is finished, because the only judge available is the same model that did the work.

Jeffy Loop is a skill for Claude Code built around a single commitment: **the decision to stop is taken away from the model.** A run ends when a shell script, holding no weights and no opinion, re-checks every claim the run has made and finds them all still true. If any check fails, the script feeds the failure back into the loop and the run continues.

This is a narrow idea, and most of the system follows from it. If a shell script has to verify the claims, the claims must be written somewhere a script can read. If the script re-runs the project's test command, that command must be recorded in a fixed place and must not be allowed to lie. If findings are to be re-checked, they must have been reproduced in the first place rather than asserted. The rules described in this part are not a collection of good practices. They are the consequences of taking the stopping decision away from the model and giving it to a program.

## 13. Anatomy of a run

A Jeffy run has four files, one prompt, one hook, and a commit after every iteration.

**The state files** live at the root of the project being worked on, and they are the loop's memory. `PLAN.md` holds the goal, the operating envelope, the method, the severity rubric, the verify command, the surface inventory, and the definition of done. `BACKLOG.md` is a ledger of open work, deliberately not a narrative: tasks are one-line items with an acceptance check, and a completed task is deleted from the ledger rather than annotated. `JOURNAL.md` is append-only, one entry per iteration, following a fixed heading grammar so that a script can parse it. A fourth file, `.claude/jeffy-loop.local.md`, is transient session state holding the current iteration number, the budget, and the session identity. It is deleted when the run ends and is never committed.

**The iteration prompt** is a single line of text stored on disk. It contains the entire discipline of the loop: how to audit, when to file a finding, how severity is judged, what the journal entry must contain, what convergence requires. It is not injected into the conversation once and left to decay. It is re-read from disk and re-fed at the end of every single turn.

**The Stop hook** is the engine. Claude Code lets a user register a script that runs when the assistant finishes a turn, and that script can refuse to let the turn end. Jeffy's hook is about 600 lines of plain shell. At every turn end it reads the state file, and while the budget remains it re-feeds the iteration prompt, which starts the next iteration. This is the mechanism that makes the loop a loop. Crucially, the hook is also the auditor: when the model attempts to declare convergence, the hook independently verifies the claim before permitting the session to end.

**The checkpoint** is a local git commit made at the end of every iteration with the message `jeffy: iter i/N task status`. Nothing is ever pushed and no branches are created. The checkpoint

serves three purposes: it is the unit the loop reverts to when an iteration breaks the build, it guarantees the next iteration starts from a clean working tree, and it leaves a reviewable history so a human can read the run backwards.

The table below summarises what each piece is and who controls it. The right-hand column is the one that matters: the components a model can influence are separated from the components it cannot.

| Component | What it is | Who controls it |
|---|---|---|
| PLAN.md | Goal, envelope, method, severity rubric, verify command, surface inventory | Model writes, script reads and checks |
| BACKLOG.md | One-line open tasks, each with an acceptance check | Model writes, script checks emptiness |
| JOURNAL.md | Append-only record, one entry per iteration, fixed grammar | Model writes, script parses |
| Iteration prompt | The entire discipline of the loop, re-fed every turn | Fixed on disk, model cannot edit |
| Stop hook | Shell script that re-feeds the loop and audits the stop | Fixed on disk, model cannot edit |
| Checkpoint commit | Local git commit ending each iteration | Made by the model, verified by the script |
| Verify command | The project's own test suite | Chosen at setup, re-run independently by the script |

A run therefore looks like this. The user types `/jeffy 10` in a project. The skill runs nine pre-flight checks, copies the state file templates in if they are missing, writes the session state, and begins iteration one. Because a fresh project has an empty backlog, iteration one is always consumed by an audit that generates the work. Every iteration after that executes exactly one task, never a batch, verifies it, runs the project's own test command, writes a journal entry, and commits. The hook re-feeds the prompt. The run ends when the budget is exhausted, when a hard blocker is hit, or when convergence is earned and verified.

## 14. The rules, and why each one exists

### Evidence before filing

A finding does not exist until the loop has reproduced it. Not "this looks wrong," but a command that was run, with its real output, recorded before the finding is written down.

The reason is that a language model asked to find defects will find them, because that is what it was asked to do. Plausibility is cheap and abundant. Reproduction is expensive and scarce, and the expense is the point: writing the reproduction is where the model usually discovers the bug is

not real. The rule converts an unfalsifiable opinion into a claim that a shell script can re-run later, which is what the whole architecture depends on.

### The operating envelope

Before any finding can be filed, `PLAN.md` must state the project's real input surfaces: what this software is expected to receive, from whom, under what trust assumptions. Severity is then judged against that envelope rather than in the abstract. A defect reachable only by input the project never claims to accept is a Low at most, no matter how dramatic it looks.

Without an envelope, severity is a mood. An agent can call anything critical, and a list of critical findings that are mostly hypothetical is worse than no list, because it destroys the reader's ability to triage. The envelope gives severity a denominator.

### The surface inventory

The first audit must map the project's entire public surface into a checkbox table, one row per public module or entry-point group, and probe every row at least shallowly before investigating any row deeply. A row is only ticked with the commit hash it was swept at and one line naming what the sweep exercised. If the code behind a ticked row later changes, the row goes back to unticked.

This exists to stop a specific dishonesty. When an agent reports "no findings," the reader hears "this project is clean." What the agent usually means is "I did not find anything where I looked." The inventory forces the second statement to be visible, and it forces breadth before depth, so the worst defect in a project surfaces in the first audit rather than the sixth.

The rule has a sharp corollary for code that computes values. Sweeping such a surface requires a known-answer or invariant check for each function family, never a probe that merely confirms the code runs without crashing. Wrong numbers returned without complaint survive every liveness probe ever written. Similarly, every documented parameter must be exercised at two or more values that must change the output, and a documented parameter whose value changes nothing is itself a finding rather than a pass.

### The verify gate, and the refusal that protects it

`PLAN.md` records the project's own verification command, typically its test suite. After every task, the loop runs it. If it fails and it passed at the last checkpoint, the loop concludes that this iteration broke the project: it reverts the working tree to the last checkpoint, marks the task blocked with the failure output, and records what happened.

One refusal protects this gate and deserves its own explanation, because it is the least obvious rule in the system and the most consequential. Jeffy refuses a verify command whose pipeline ends in a pager or truncator such as `head`, `tail`, `less`, `more` or `cat`. The reason is a property of shell pipelines: the exit status of a pipeline is the exit status of its last command. A user who writes `pytest | tail -1` to keep the output short has written a command that reports success even when every test fails, because `tail` succeeded at printing the last line. A loop trusting that

command would converge over a red suite, having verified nothing. The refusal fires when the run is launched and again at the gate.

### The adversarial evaluator

Before the loop may claim convergence, it must spawn a fresh sub-agent as an adversarial evaluator. That sub-agent carries none of the run's conversation. It is instructed to assume the work is broken until proven otherwise and not to praise. It reads the plan and the backlog, reviews the run's changes with git, re-runs the verify command and the acceptance checks of the tasks closed during the run, reports their real output, and hunts for missed findings in the code the run touched. It must return PASS. A rejection files its reasons as new backlog tasks and the run continues; a second rejection ends the run as a hard blocker.

The honest description of what this buys, and what it does not, matters. The evaluator is the same model. A blind spot in the weights sits on both sides of the table, so this is not independence. What it buys is fresh context. It never sees the reasoning that produced the fix, only the artifacts and an instruction to break them. That defeats the specific failure where a model talks itself into a conclusion over many iterations and then grades its own conclusion favourably. It does nothing about a blind spot both instances share. Section 18 gives a documented case where the evaluator passed a finding that was, in fact, wrong.

There is a further discipline about timing. The gate fires while its verdict can still be acted upon, meaning the iteration where the backlog first empties with a clean audit and at least three iterations of budget left, rather than at the very end. A rejection that arrives with no budget remaining files work nobody can do.

### The machine-checked stop

This is the load-bearing component, and it is not a model. The Stop hook refuses the converged stop unless every one of the following holds: the backlog is genuinely empty, the recorded convergence commit still certifies the current tree, no inventory row is unswept, the closing journal entry carries an evaluator verdict, and the project's own verify command exits green when the hook itself re-runs it. A failed check does not produce a warning. It re-feeds the loop with the evidence, and the run continues.

The asymmetry here is the entire design. The model can claim anything it likes. The claim only becomes an ending if a program that cannot be persuaded agrees.

It is worth being exact about how thin, and how important, the difference from the vendor implementations in Part I section 6 is. Jeffy also uses a completion phrase. The model signals that it believes the run is over by emitting a specific string, exactly as the Anthropic and Cursor plugins do. The difference is what happens next. In those implementations the phrase **is** the stopping condition: matching it ends the run. In Jeffy the phrase is only a *request* to stop, and the hook then goes and checks. It re-reads the ledger to confirm it is empty, re-reads the inventory to confirm no row is unswept, confirms the recorded convergence commit still describes the current tree, confirms an evaluator verdict is on the record, and re-runs the project's test command itself. Any one of those failing turns the declaration into another iteration.

Both designs ask the model to say when it is done. Only one of them treats the answer as a claim to be audited rather than a fact to be accepted. That single inversion is what this project consists of, and everything else in this part exists to give the auditing program something it can actually check.

The engine is itself held to 125 behavioural checks on each of three continuous integration legs, Linux, Windows and macOS, with a shellcheck lint pass riding the Linux leg on top.

### Closeout, convergence and the ratchet

Two smaller rules keep runs terminating. **Closeout** says that once a full fresh-evidence audit has scored zero High and zero Medium findings, the run stops auditing for the remainder of the run and simply finishes the ledger. Without it, a project with a long tail of minor issues would loop forever: each emptied backlog triggers another audit that files another minor issue. **The ratchet** says that if a previous run converged and nothing but the loop's own state files has changed since, the next run verifies that and stops immediately rather than re-auditing an unchanged tree.

Convergence itself requires all of the following simultaneously: a full fresh-evidence audit this run scoring zero High and zero Medium within the envelope, no unswept inventory row, an empty ledger where every finding including every Low is fixed, declined with a genuine reason, or blocked with its reason on record, a green verify command, and the evaluator's PASS.

## 15. Enhance mode

The default loop hunts for defects. Some valuable work is not a defect: a capability a project stops short of, a workflow that takes five steps where a peer tool takes one, a platform the installers support but nothing exercises. `/jeffy 8 enhance <topic>` points the same machinery at that work, with an opportunity audit replacing the defect audit.

Two refusals define the mode. An enhance run with no topic is refused outright, because an unbounded instruction to make things better is precisely the invented work the operating envelope exists to prevent. And the two modes never share state files, because they rank work differently and their ledgers must not mix. Everything else is identical: the same Stop hook, the same verify gate, the same checkpoints, the same evaluator. Convergence is earned the same way or not at all.

## 16. What this is trying to achieve

The goal is not to make an agent that writes better code. It is to make an agent whose report you can act on without repeating its work.

That is a claim about trust, and trust in this setting is mechanical rather than emotional. When a run reports that it found four data-loss bugs and fixed them, the value of that report depends entirely on what would have had to be true for it to be false. In an ordinary agent loop, the answer is very little: the model could have been wrong, or persuaded itself, and nothing would have caught it. In a Jeffy run, a false report requires the reproduction to have run and produced its output, the project's own test suite to have passed under a hook that re-ran it independently, a

fresh-context adversary to have failed to break it, and a shell script to have re-verified every structural claim. None of that makes the report certainly true. All of it makes the report expensive to fake, and the difference between a claim that is cheap to make and a claim that is expensive to fake is most of what practical trust consists of.

The second goal is subtler and more important for the reader deciding whether any of this generalises. The engine ships no language-specific analyser, no ruleset, and no per-ecosystem plugin. It works entirely from what a project already has, which is its own test suite and its own verification command. That is why the same skill runs against a Rust command-line tool, a Ruby linter and a C++ parser without modification, and it is why the method rather than the tooling is the transferable part.

## 17. The receipts

Testing a tool against its own codebase proves very little. Jeffy has therefore been run against widely used open-source projects with no connection to the project, using a local clone, with nothing pushed upstream without a filed issue or pull request. Sixteen of those runs converged, across eight languages: Python, JavaScript, Java, C#, C++, Go, Rust and Ruby. Every run is published in full [25].

| Project | Language | Iterations | Upstream outcome |
|---|---|---|---|
| fasthttp | Go | 58 | fix merged |
| bat | Rust | 10 | fix merged |
| chalk | JavaScript | 8 | fix merged |
| dayjs | JavaScript | 74 | pull request open |
| python-dotenv | Python | 73 | pull request open |
| PyPortfolioOpt | Python | 58 | pull request open |
| jsoncpp | C++ | 10 | pull request open |
| yfinance | Python | 9 | pull request open |
| quantstats | Python | 40 | issue filed |
| records | Python | 7 | issue filed |
| mustache.js | JavaScript | 11 | issue filed |
| Spectre.Console | C# | 8 | issue filed |
| ta | Python | 64 | none filed |
| speedtest-cli | Python | 5 | none filed |
| gson | Java | 2 | none filed |
| RuboCop | Ruby | 7 | none filed |

Each receipt in the repository is a directory containing the run's journal, the backlog it wrote, the patch it produced, and a link to the upstream issue or pull request where one was filed. The findings are varied in a way that is itself evidence: a security flag that did nothing when output was piped, a documented build configuration that had never compiled on one compiler, a Content-Length value no parser should accept becoming a wrong number, a panel header dropped rather than truncated.

The clearest single example is also the shortest. In `kennethreitz/records`, a Python database library, `pytest` reported 31 tests passing. At that same commit, four separate High-severity defects were live, all of them data loss: an `INSERT` committed nothing and the rows vanished silently on close, bulk inserts did the same, a bare exception handler swallowed every transaction failure so callers believed failed writes had succeeded, and every query leaked a pooled connection. The suite stayed green through all four because its fixture used an in-memory database sharing a single connection, where uncommitted state is always visible and nothing ever crosses a real connection boundary. The gate was measuring the wrong world. That is the failure mode this entire system exists to catch, and no amount of running the existing tests would ever have caught it.

Two entries in that table are worth as much as the dramatic ones, for the opposite reason. The `RuboCop` run swept every department of a mature Ruby linter, individually re-proved the last twenty upstream commits, filed nothing, and changed not a single line. The `gson` run took two iterations, found one Low-severity issue, priced it, declined it, and stopped. A method that always finds something is not measuring the code; it is measuring its own eagerness. The null results are the control group, and they are published in the same table as the successes rather than quietly omitted.

The table also carries one deliberate omission and one deliberate failure. `PapaParse` appears in the repository as an audit rather than a converged run, because its loop conversion waits on four open upstream pull requests. And `python-dotenv`, the longest entry at seventy-three iterations across eight separate runs, was published on the project's front page as **not converged** for a long period, through four runs and twenty-five findings, because every full audit it ran kept finding something new. That record was kept rather than deleted when the project finally did converge, on the principle that a method which only publishes its successes is not being measured.

Three findings have been merged upstream by maintainers who had no stake in this project: in `bat`, in `chalk`, and in `fasthttp`. Those merges matter more than any convergence flag in the repository, for a reason section 18 makes explicit.

### The greenfield arm: two builds judged by suites the loop did not write

Every receipt above is brownfield: an existing project arrived with an existing test suite, so the gate predated the loop. Section 18 records the residual weakness even so - the loop writes many of the tests that certify the loop. Greenfield is that weakness at its maximum. With no pre-existing code there is no pre-existing oracle, and a naive greenfield run would author one hundred percent of its own gate and converge against homework it set itself. The pre-registered answer is to hand the judgement to a suite the loop cannot edit. Two targets were chosen to be different oracle classes, because that difference is the argument rather than an incidental detail:

| | Corpus author | Judge | Code author |
|---|---|---|---|
| Brownfield (the sixteen above) | external | external | external |
| Target 1, TOML decoder | external | external | the loop |
| Target 2, gitignore matcher | the loop | external | the loop |

For each target, `PLAN.md` was committed before iteration 1 - the goal, the operating envelope, the surface inventory, and a Verify command naming the external judge - so `git log` proves the gate predates the work. One deliberate deviation from this project's own documentation is recorded here rather than discovered: `BACKLOG.md` shipped empty, against the Scoped-mode advice to seed finite tasks, so that the task decomposition would be the loop's work and not the author's. The engine was not modified for either target, and no run was intervened in.

**Target 1** is a TOML v1.0.0 decoder in Rust, judged by `toml-test` v2.2.0 [26]: 205 valid cases, 474 invalid, 679 assertions. The inventory is external too - the suite's own test groups supply the 17 surface-inventory rows, which makes this target stricter than any of the sixteen on that axis, since in brownfield the loop writes the inventory from the code it reads. The zero measurement before iteration 1 was the suite failing on the first case because the binary did not exist. Eleven iterations later, in a single run, the suite reported every assertion passing and the adversarial evaluator passed the declaration on its first invocation, after twenty hostile probes of its own. One rule from the pre-registration matters for reading the numbers: valid and invalid counts are always reported separately, never as one fraction, because a stub that rejects every input scores 474 of 679 assertions while decoding nothing - a single collapsed pass rate is gameable by refusal.

**Target 2** is a `.gitignore` matcher in Rust with no packaged suite in existence, so the judge is the real `git` binary [27]: a differential harness materializes a repository per case in an isolated configuration and compares the matcher's verdict against `git check-ignore -v` on every query, with an oracle error never readable as agreement. The 12 inventory rows were frozen from `gitignore(5)` before iteration 1. The corpus, though, is the run's own - that is the deliberately weaker oracle class, and section 18 weighs it - mitigated by freezing: the corpus only ever grew, 53 cases at the freeze to 106 at convergence, and its size is printed beside every pass count. The final position is 106 cases, 300 queries, zero disagreements against git 2.50.1 on Windows, and the verdicts are environment-specific by construction: the oracle ran on NTFS with `core.ignoreCase` true, which is how the run came to implement git's actual casefold semantics rather than the manpage's.

| Target | Spec | Judge | Final position | Rows swept | Iterations | Runs |
|---|---|---|---|---|---|---|
| TOML decoder, Rust | TOML v1.0.0 | `toml-test` v2.2.0 | 205 of 205 valid, 474 of 474 invalid | 17 of 17 | 11 | 1 |

| Target | Spec | Judge | Final position | Rows swept | Iterations | Runs |
|---|---|---|---|---|---|---|
| gitignore matcher, Rust | `gitignore(5)` | `git check-ignore` 2.50.1 | 106 cases, 300 queries, 0 disagreements | 12 of 12 | 42 | 5 |

The gitignore arc is the part a skeptic should read first, because it is a record of the gate refusing. The frozen corpus was fully green from the first iteration of the second run; everything after that - nearly four runs of it - was the adversarial evaluator probing beyond the corpus and declining to countersign. It was invoked eight times across runs 2 through 5 and rejected seven, each rejection substantiated by disagreements reproduced through the run's own harness, and its thirteen filed findings marched steadily outward from matcher semantics into territory that is genuinely obscure: `wildmatch.c`'s escaped-slash clause, git's own `isspace` accepting exactly four bytes, NTFS folding names the matcher compared byte-wise, 8.3 short-name aliases whose byte length differs from the directory they name, and the layer boundary where Win32 normalization strips trailing dots that git's extended-length file opens never see. Three runs ended blocked, out of evaluator invocations, and each block is published as the receipt it is. Once, the loop refuted its own gate: the fourth rejection arrived with three reasons, the loop reproduced two and disproved the third with direct oracle evidence, filed the two and recorded the refutation. The state files - journal, plan, backlog - ship inside both repositories [26][27], which reverses this project's own rule of keeping its development journal out of the published tree: for a brownfield fix the upstream diff is the evidence, and for a greenfield build the process is.

The claim, stated narrowly: the same engine, unmodified, in Scoped mode, converged on a build from an empty directory whose completeness was decided by a judge it did not write. Four things are explicitly not claimed. Not a completion rate - two chosen targets are not a sample, and this paper states no percentage. Not an answer to Huntley's ninety percent, which is a self-assessed rate over many projects. Not a statement about delivery quality - conformance says the decoder matches the spec, nothing about whether anyone would want the API. And not invention, for the reason section 18 now states.

## 18. Honest limits

**The verifier shares the generator's blind spots.** The adversarial evaluator is the same model with fresh context. It defeats self-persuasion; it does not defeat a defect in the training data. There is a documented case. In the `python-dotenv` run, the loop filed a finding claiming that variable-default syntax contradicted the library's documented precedence. The evaluator passed it. Only a later verification, performed by agents working from a clean clone that had never seen the loop's tree, established that the premise was false: the documentation defined the behaviour the code exhibited, so the contradiction claimed did not exist. The finding is retracted in the published receipt rather than quietly dropped.

**The loop writes many of the tests that certify the loop.** In that same project the suite grew from 220 tests to 511, and 291 of those are the loop's own. They are falsifiable, and restoring the

original source files fails exactly the tests that pin the run's fixes. But they were written by the same process that wrote the fixes, and a reader should discount them accordingly. The greenfield arm in section 17 is the pre-registered reply to this limit, and it carries two limits of its own, stated next.

**A frozen self-authored corpus is not an external corpus.** The gitignore target's judge is git itself, but the questions put to that judge were generated by the run, so the run controlled the denominator. The mitigations are real - the corpus was frozen at run start, never shrank across five runs while growing from 53 cases to 106, and its size is printed beside every pass count so a reader can see which kind of number they are looking at. They are still mitigations. The TOML result rests on 679 assertions authored by people with no connection to this project; the gitignore result rests on an externally adjudicated corpus whose shape the loop chose, and the two should be weighed accordingly.

**A model that has read many TOML parsers is not building from nothing.** Both greenfield targets are precisely specified, heavily documented formats with many public implementations in any plausible training set. The greenfield claim is about stopping discipline under an external judge - the engine kept working until a suite it could not edit said finished, and its own gate refused the declaration seven times after the frozen corpus was already green. It is not a claim about invention. A novel specification with no public implementations would be a different and harder test, and it has not been run.

**A known-answer check cannot argue about which answer the context demands.** This is the sharpest boundary found so far, and it emerged from a converged run. In `quantstats`, a financial statistics library, a later manual audit found six defects the loop did not, all still present at the converged tree. They share one shape: they are convention defects, meaning internally consistent numbers that are wrong in context. A bet-sizing function implemented the textbook formula faithfully, and the textbook formula produces a result unchanged when every return is halved, which no capital-allocation recommendation can be. A ratio was returned per-period and printed directly beneath an annualised figure, differing by a factor of the square root of 252. The loop's probes validated the same formula the code implemented, certifying the convention instead of testing it. A known-answer probe can verify that a formula was computed correctly. It cannot adjudicate which formula the situation calls for. That requires a domain expert.

**Convergence is a statement about a contract, not a certificate of correctness.** It says that under a declared envelope and sweep contract, one full fresh-evidence audit found nothing High or Medium on a fully swept surface, and an adversarial reviewer failed to break that result. It does not say there is nothing left.

**Platform monoculture hides defects the same way test monoculture does.** Sixteen converged runs across eight languages executed on Windows and on Linux. Not one of them ever ran on macOS. When a macOS continuous integration leg was finally added, it went red on its first run and named a real defect in the engine: the verify gate probed for a command-line utility before running any of its checks, and stock macOS does not ship that utility, so on every Mac the gate skipped itself entirely and silently. A macOS user whose verify command ended in a truncating pipe could therefore have converged over a red suite with no diagnostic, which is

precisely the failure the truncator refusal exists to prevent. No number of additional runs on Windows or Linux would have found it, because both of those platforms provide the utility and so both take the working path. Repetition is not independence, whether the thing repeated is a model, a test fixture, or an operating system.

**The strongest available check is the one with no stake in the result.** This is why three merged upstream pull requests carry more evidential weight than sixteen convergence declarations. A maintainer reviewing a patch against their own library shares none of the loop's priors, none of its context, and none of its incentives. Every other check in this document is, ultimately, the system grading itself with progressively better discipline. Only the maintainer is genuinely outside it.

# Glossary

Terms are defined here in the order a newcomer is likely to meet them.

**Large language model.** A system trained to predict text, which in practice can follow instructions, write code and explain its reasoning. On its own it produces text and nothing else.

**Context window.** The finite amount of text a model can consider at once, holding the conversation, the instructions, and any file contents or tool output loaded into it. It is the scarcest resource in agent design, and quality degrades as it fills.

**Tool call.** A structured request from the model for something to happen outside itself, such as reading a file, running a command or searching the web. The result is returned to the model as text.

**Agent.** A model placed in a loop with tools, so that it can act, observe the result, and act again without a human between each step.

**Harness.** The surrounding program that runs the agent: it holds the loop, supplies the tools, manages the context, and decides when to stop.

**Workflow.** A system where models and tools are orchestrated along predefined code paths. Distinguished from an agent, where the model itself decides what to do next.

**Oracle.** Anything outside the model that can settle whether an answer is right. A test suite, a compiler, a mathematical identity, a specification, or a human expert. Coding is a favourable domain for agents mainly because it has cheap oracles.

**Ground truth.** Information about the real state of the world, obtained from the environment rather than generated by the model. A passing test is ground truth; the model's belief that the code works is not.

**Hook.** A script the harness runs at a fixed point in the agent's lifecycle. Unlike written instructions, which the model may or may not follow, a hook always executes.

**Stop hook.** A hook that runs when the agent tries to finish its turn, and which may refuse to let it finish. This is what turns a single-shot session into a loop.

**Subagent.** A second agent instance with its own separate context window, given a narrow task and returning only its conclusion. Used to explore without polluting the main context, or to review work without having seen the reasoning that produced it.

**Evaluator.** A model instance whose job is to judge another's output rather than produce work of its own.

**Adversarial evaluator.** An evaluator instructed to assume the work is broken and to try to break it, rather than to assess it neutrally.

**Iteration.** One pass through the loop: one unit of work, its verification, and its record.

**Operating envelope.** A written statement of what a piece of software is expected to receive, from whom, and under what assumptions. Severity of a defect is judged relative to it.

**Surface inventory.** A checklist of every public part of a project, used to ensure an audit covers the whole surface before investigating any part of it deeply, and to make clear which parts were examined.

**Acceptance check.** A specific command, recorded with a task, that determines whether that task is genuinely complete.

**Verify command.** The project's own overall gate, usually its test suite, run after every change to confirm nothing has broken.

**Checkpoint.** A commit made automatically at the end of each iteration, serving as the point the loop reverts to if the next change breaks the build.

**Convergence.** The loop's term for a finished run: a state in which a fresh audit found nothing significant on a fully examined surface, no work remains open, the project's tests pass, an adversarial reviewer countersigned, and a shell script independently re-verified all of it.

**Receipt.** A published record of one run against a real project, containing the journal, the resulting patch, and any upstream issue or pull request, so the claims can be checked by a reader.

# References

All sources accessed 4 August 2026. Where a publication date is contested or absent, that is stated rather than resolved silently.


1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. "ReAct: Synergizing Reasoning and Acting in Language Models." arXiv:2210.03629, 6 October 2022. Published at ICLR 2023. https://arxiv.org/abs/2210.03629
2. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. "Reflexion: Language Agents with Verbal Reinforcement Learning." arXiv:2303.11366, 20 March 2023. https://arxiv.org/abs/2303.11366
3. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., and others. "Self-Refine: Iterative Refinement with Self-Feedback." arXiv:2303.17651, 30 March 2023. https://arxiv.org/abs/2303.17651
4. Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. "Large Language Models Cannot Self-Correct Reasoning Yet." arXiv:2310.01798, 3 October 2023. Published at ICLR 2024. https://arxiv.org/abs/2310.01798
5. Valmeekam, K., Marquez, M., and Kambhampati, S. "Can Large Language Models Really Improve by Self-critiquing Their Own Plans?" arXiv:2310.08118, 12 October 2023. https://arxiv.org/abs/2310.08118
6. Richards, T. B., and contributors. AutoGPT, Significant Gravitas. Repository created 16 March 2023; first tagged release 12 April 2023. https://github.com/Significant-Gravitas/AutoGPT . The README warning on continuous mode and the issue reports cited in section 3 are in that repository, notably issues 920 and 3644, both closed as not planned. The widely cited release date of 30 March 2023 has no corresponding repository artefact.
7. Nakajima, Y. "Task-driven Autonomous Agent Utilizing GPT-4, Pinecone, and LangChain for Diverse Applications." 28 March 2023. https://yoheinakajima.com/task-driven-autonomous-agent-utilizing-gpt-4-pinecone-and-langchain-for-diverse-applications/ . Code published as BabyAGI, repository created 3 April 2023. https://github.com/yoheinakajima/babyagi
8. Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., and Scialom, T. "GAIA: a benchmark for General AI Assistants." arXiv:2311.12983, 21 November 2023. https://arxiv.org/abs/2311.12983
9. Liu, X., and others. "AgentBench: Evaluating LLMs as Agents." arXiv:2308.03688, 7 August 2023. Published at ICLR 2024. https://arxiv.org/abs/2308.03688
10. Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" arXiv:2310.06770, 10 October 2023. Published at ICLR 2024. https://arxiv.org/abs/2310.06770
11. Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering." arXiv:2405.15793, 6 May 2024. https://arxiv.org/abs/2405.15793
12. Gauthier, P. Aider. Repository created 9 May 2023. https://github.com/Aider-AI/aider . Documentation on linting and testing at https://aider.chat/docs/usage/lint-test.html and on git integration at https://aider.chat/docs/git.html
13. OpenAI. "Why we no longer evaluate SWE-bench Verified." 23 February 2026. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/
14. Huntley, G. "Ralph Wiggum as a 'software engineer'." 14 July 2025. https://ghuntley.com/ralph/ . Secondary sources place the technique's origin earlier in 2025, variously in February, May and June; only the publication date of this post is independently verifiable.
15. Farr, C. ralph-playbook. Repository created 9 January 2026. https://github.com/ClaytonFarr/ralph-playbook . Frequently miscited as Huntley's own work; the repository under his account is a fork of this one.

16. Huntley, G. "everything is a ralph loop." 17 January 2026. https://ghuntley.com/loop/
17. Anthropic. ralph-loop plugin, claude-plugins-official. Originally named ralph-wiggum and renamed 6 January 2026. https://github.com/anthropics/claude-plugins-official . The termination behaviour cited in section 6 was read from the plugin's Stop hook implementation.
18. OpenAI. "Addendum to o3 and o4-mini system card: Codex." 16 May 2025. Section 2.3, "Falsely claiming to have completed a task that it did not complete." https://cdn.openai.com/pdf/8df7697b-c1b2-4222-be00-1fd3298f351d/codex_system_card.pdf
19. Advani, L. "From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents." arXiv:2606.09863, 1 June 2026. https://arxiv.org/abs/2606.09863
20. Anthropic. "Building effective agents." 19 December 2024. https://www.anthropic.com/engineering/building-effective-agents
21. Anthropic. "Best practices for Claude Code." Living documentation carrying no publication date; accessed 4 August 2026. https://code.claude.com/docs/en/best-practices
22. Zhang, B., Lazuka, K., and Murag, M. "Equipping agents for the real world with Agent Skills." Anthropic, 16 October 2025. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
23. Anthropic Applied AI team. "Effective context engineering for AI agents." 29 September 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
24. Rajasekaran, P. "Harness design for long-running application development." Anthropic, 24 March 2026. https://www.anthropic.com/engineering/harness-design-long-running-apps
25. Jeffy Loop. Source, receipts and validator. https://github.com/lenamonj/jeffy-loop . Every figure in Part III was recomputed from that repository at the commit current on 4 August 2026 rather than quoted from its prose.
26. Jeffy Loop greenfield receipt: TOML 1.0 decoder, judged by `toml-test` v2.2.0. https://github.com/lenamonj/jeffy-greenfield-toml . The pre-registration, journal, backlog and every iteration commit ship in the repository; the final suite output was re-derived from a fresh clone at the converged commit.
27. Jeffy Loop greenfield receipt: gitignore matcher, judged differentially against `git check-ignore`. https://github.com/lenamonj/jeffy-greenfield-gitignore . The pre-registration, the frozen corpus, and the journals of all five runs, including the three that ended blocked, ship in the repository; the final differential output was re-derived from a fresh clone at the converged commit.

---

The source, every receipt cited in Part III, and the validator that holds the engine to its 125 checks:

**https://github.com/lenamonj/jeffy-loop**